S. No	Communication Type	Frequency	Channel / Tool	Participants	Purpose
1	Team Standup	Daily	Discord / Slack	Core Project Team	Sync daily progress, remove engineering blockers, and track app.py integration progress.
2	Progress Update	Weekly	GitHub Pull Requests	Core Project Team	Review feature merges, check validation scripts, and update the repository milestone logs.
3	Issue / Bug Discussion	As Needed	WhatsApp / Zoom	Technical Leads	Hotfix prediction runtime exceptions, handle missing array bounds, or resolve joblib loading errors.
4	Stakeholder Review	Bi-Weekly	Microsoft Teams	Team & Evaluators	Present early model testing loops, gather feedback on glassmorphic UI layout, and show progress.
5	Final Demo Rehearsal	Once	Google Meet	Full Team	Conduct an end-to-end dry run of the dashboard, verify live Ngrok public URL stability, and practice the presentation script.

6	Post-Mortem Sign-off	Project Close	Google Docs / Email	Full Team & Lead	Consolidate the final delivery reports, verify all documentation, and hand over project assets.

Communication

Date	13 JULY 2026
Team ID	XXXX
Project Name	Comprehensive measure of well-being
Maximum Marks	1 Mark

1. Communication Plan

Effective communication is essential for a successful project demonstration. This document outlines the communication strategy used within the team and with stakeholders throughout the project lifecycle, including how updates, issues, and feedback were managed.

2. Communication Challenges & Resolutions

This provides the professional technical context for the challenges faced by your team and how you resolved them.

S.No	Challenge Faced	Resolution / Action Taken
1	Asynchronous API Integration Delays: Misalignment between frontend JavaScript developers and backend Python engineers regarding the JSON layout structure for the /simulate endpoint payload.	Standardized an explicit API schema contract inside shared documentation. Built mock JSON files upfront so the UI team could write independent fetch() routines without waiting on backend endpoints.
2	Merge Conflicts & Version Drift: Simultaneous edits to the predictive wrapper logic in app.py caused major repository conflicts on GitHub, stalling deployment pipelines.	Implemented a strict modular feature-branch workflow. Enforced short-lived branches and established mandatory peer-review approvals before merging any core architectural modifications into the main branch.
3	Unstable Remote Collaboration Testing: Team members working remotely could not access or interactively test the locally hosted Flask server running on individual developer machines.	Integrated the Ngrok tunnel gateway into a utility file (run_public.py). This streamed a reliable, secure public endpoint to all decentralized teammates instantly, eliminating environmental friction during remote testing windows.